



TECHNICAL DOCUMENTATION

How the spending boundary is actually enforced.

Architecture, the authority model, the state machine,
and an honest account of what is real and what is
standing in.

Sujal Negi

Student, IIITDM Kurnool

sujalnegi.tech

What the system is

Limen is an AI procurement agent whose spending authority is held by a smart contract rather than by the application that runs it.

The agent reads a purchase request in plain language, screens a supplier catalogue against hard constraints, negotiates against reservation prices it cannot see, and returns one recommended deal. Everything after that point requires people, and the money requires three of them working at separate desks.

The claim, stated precisely

A conventional agent is constrained by checks inside its own process. Those checks hold while the process behaves and are the first thing to fail when it does not. Here the per deal ceiling is contract state on an EVM chain. The agent may construct and send a transaction that exceeds it. The network rejects that transaction, and the rejection does not depend on the correctness of any code in this repository.

The property that carries it

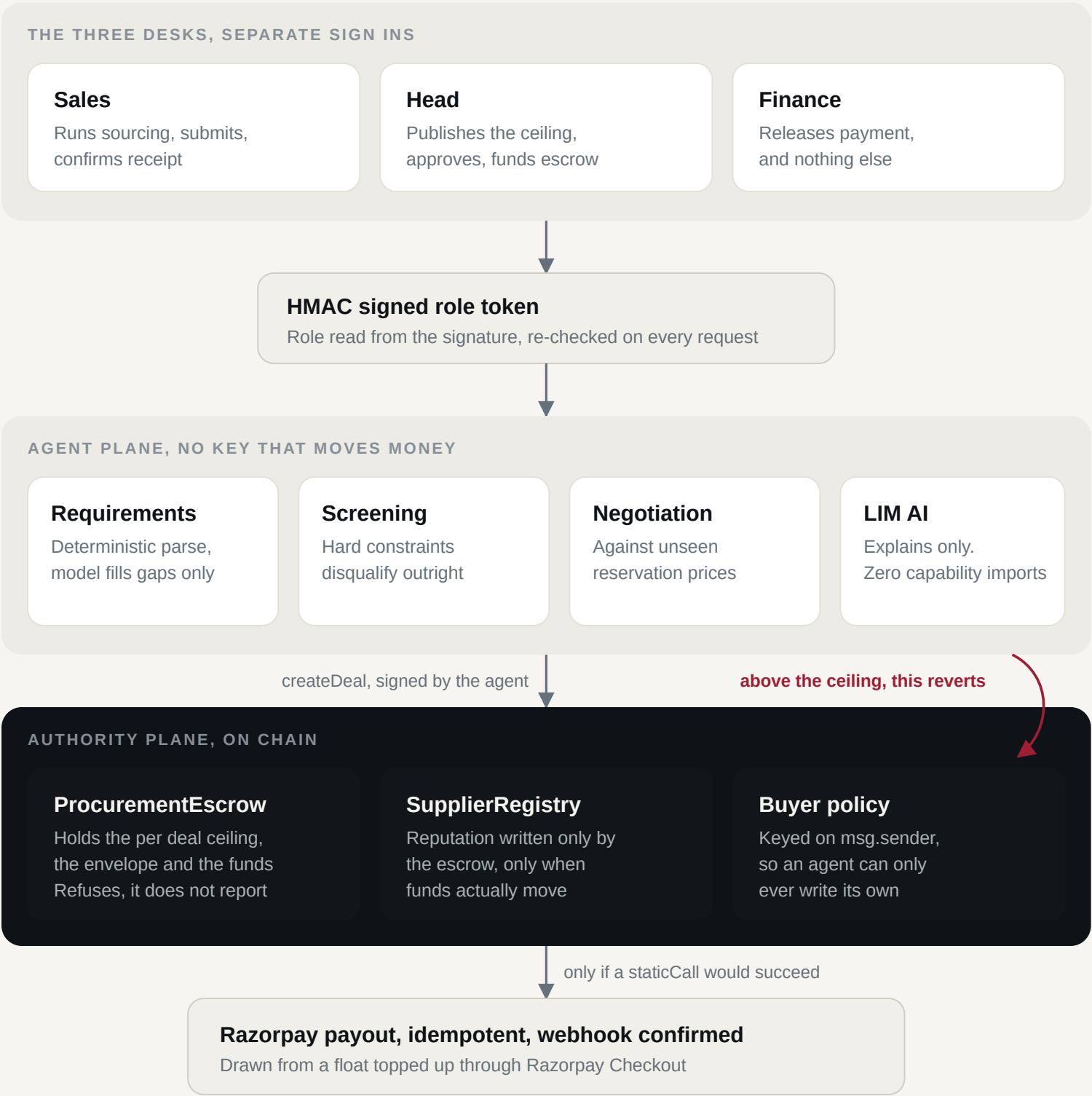
The policy record is keyed on `msg.sender`. An agent writing a spending policy can therefore only ever write its own, never the buyer's. Privilege escalation is not a check that could be forgotten. It is not expressible in the contract's state model.

Stack

LAYER	IMPLEMENTATION
Contracts	Solidity 0.8.24, compiled in process with solc-js
Chain	In process EVM by default, any JSON-RPC network via <code>RPC_URL</code>
Server	Node and Express, no ORM, no framework beyond routing
State	Postgres, with an in memory adapter behind the same interface
Interface	React 18 and Vite, hand written CSS design system
Payments	Razorpay Checkout in, Razorpay Payouts out

Architecture

Three planes. The agent plane reasons and never holds a key that can move money. The authority plane is on chain and holds the ceiling. The settlement plane moves real money and is gated by the authority plane before it is ever called.



The gate that matters

Before the payment rail is contacted at all, the server runs the release as a `staticCall` against the real contract. If the policy would refuse the payment, it refuses here, and the external call is never made. The test that pins this asserts a count: contract refuses, Razorpay calls equal zero.

Where each workspace's money lives

Every workspace derives its own buyer wallet from a mnemonic at a stable index, so two workspaces cannot spend from one balance and a workspace resolves to the same address across restarts. That matters more than it sounds: a policy the head published yesterday has to still be the policy the contract reads today, and a fresh key per boot would silently orphan it.

The purchase state machine

State is derived, never stored. A stored status is a second source of truth that drifts from the facts it summarises, so the state is computed from what is actually recorded: whether a recommendation exists, whether a head approval exists, whether the escrow holds funds, what the payment rail last reported.

```
// twelve states, derived
DRAFT
  to AI_COMPLETED      agent returned a recommendation
  to SALES_REVIEW      sales submitted it
  to HEAD_APPROVAL     sales sent it up
  to APPROVED          head approved           head only
  to FUNDED            escrow funded           head only
  to PAYMENT_READY     supplier attested,
                      buyer confirmed receipt
  to PAYMENT_PROCESSING payout instructed     finance only
  to SETTLED           webhook confirmed

REJECTED, FAILED, REVERSED are terminal branches.
```

Two guards on every transition

Each route asks the same two questions before doing anything. Is this the right step from where the purchase currently stands, and does the caller hold the permission for it. Neither question is answerable from the browser, which is the point.

The commercial fingerprint

An approval binds to the terms it was given, not to the purchase in general. The document is rebuilt and re-hashed at every step that moves money, and the hash is compared against the one recorded at approval. If the terms changed after approval, funding and payment both refuse and it goes back to the head.

The fingerprint deliberately excludes the authority block, so publishing a policy does not invalidate an approval that was made before it.

Concurrency

The workspace is one document with a version column. Writes take a per workspace lock and save under optimistic concurrency, so two desks acting at the same moment cannot silently overwrite one another. A conflict returns 409, which is the one case where the correct client behaviour is to retry rather than to correct something.

The authority model

PROPERTY	ENFORCED BY
Spending ceiling	<code>ProcurementEscrow.createDeal</code>
Agent cannot raise the buyer's ceiling	Separate policy records keyed on sender
Only the nominated agent may spend	<code>NotAuthorisedAgent</code>
Reputation cannot be forged	Registry writable only by the escrow
A role cannot be chosen by the caller	HMAC token, role read from the signature
A delivery needs two parties	<code>attestShipment</code> then <code>confirmDelivery</code>
A payment is real, not claimed	Gateway signature verified server side
The model cannot act	No capability imports in the counsel module

The capability boundary, as an import list

The module that answers questions imports nothing that can move money. Not the chain, not the payment rail, not the workflow. It is a boundary a reader can verify by looking at the top of one file, which is a stronger guarantee than a sentence in a prompt asking a model to behave.

The model may phrase, it may not compute

Every figure is calculated in ordinary code from server state and handed to the model as finished text. The rewrite is then checked in both directions: every number in the source must survive it, and any number not in the source causes the rewrite to be discarded. If either test fails, the deterministic text ships and the interface says which one is being read.

What the desk codes are not

Sign in uses a short code per desk with a throttle that doubles after five failures. This stops the wrong browser tab becoming the approver, which is the mistake that actually happens when one person walks through all three desks. It is not an identity system, there are no accounts behind it, and a real deployment replaces that module with the organisation's own sign in.

How it is verified

Four suites, because they answer different questions and the first three cannot see what the fourth sees.

SUITE	CHECKS	ANSWERS
npm test	280	Contracts, engine, documents, storage, identity, payments, in isolation
npm run sweep	154	The live HTTP surface, in the order a person actually uses it
verify:ui	65	What a real browser draws, across all three desks
verify:tier4	16	Delivery signatures, payments, contrast in both themes

Why the browser suites exist

A previous round shipped three navigation items with no screen behind them, an approver shown nothing to decide on, and a progress indicator reading a browser variable instead of the purchase. Every one of those passed the entire test suite, because a test that talks to an API cannot see an empty screen.

Notable properties under test

- A contract refusal means the payment rail is never called, asserted by call count
- No desk can perform another desk's action, in any state, in any order
- A token cannot be re-signed, reused across workspaces, or minted by the client
- A purchase, an approval and the audit trail all survive a restart, on both storage adapters
- A forged, replayed or borrowed payment signature is refused
- A model rewrite that invents a figure is refused, not only one that drops it
- An unauthenticated read cannot create a workspace, and therefore cannot evict one

Limitations, stated plainly

The value of this system is that it is honest about the difference between what it enforces and what it depicts. These are the places where it depicts.

AREA	WHERE THE LINE SITS
Delivery	Two signatures narrow a fictitious delivery from one party to two working together. It does not eliminate it. That needs an attestation from somebody with no stake in the trade.
The supplier	Counterparties are simulated agents and their keys live in this process. The contract requires the supplier's own address, which a real deployment inherits unchanged.
The catalogue	Seeded unless a directory is configured. The engine only ever cared about the schema, so connecting a real vendor list is an adapter.
Currency	Purchases are in USD, the payment rail is INR, and the conversion uses a stated constant rather than a live rate. Every converted figure names the rate, and the rate is stored on the payment record.
Identity	Desk codes, not accounts.
Signature	The e-signature records a name, a timestamp and a document hash. It is not a legally binding electronic signature.

What would change the answer

A carrier or inspector integration closes the delivery gap. An enterprise identity provider makes the separation of duties an organisational fact rather than a product feature. A live supplier directory turns a demonstration catalogue into a market. None of the three is research. All three are integrations, and the architecture is shaped so that each is a substitution rather than a rewrite.

Sujal Negi

Student, IIITDM Kurnool

sujalnegi.tech · github.com/sujal128005/limen